

CS524-A Spring 2026

Introduction to Cloud Computing

FINAL PROJECT

AI-POWERED SECURE WEB APPLICATION

Analysis Report

Name: Mohammed Abdul Haseeb

CWID: 20036044

Email: mhaseeb1@stevens.edu

CloudFront URL: <https://diq0ysy4we8cm.cloudfront.net/>

Introduction:

This project is the building of an AI-powered secure web application on AWS by connecting static website hosting, serverless backend processing, cloud storage, and monitoring into one complete workflow. The application is that text entered on the website and is sent to the backend, analyzed using NLP, and then shown back on the frontend. To make that happen, Amazon S3 was used to host the static website files, CloudFront was used to deliver the website securely over HTTPS, AWS Lambda was used as the backend compute service, and CloudWatch was used for logging and monitoring. The backend logic was built to perform text analysis tasks such as sentiment detection, entity extraction, key phrase identification, and readability scoring, while also saving the generated output in Amazon S3 as JSON files.

This report shows implementation to show how the project from the initial setup stage to final deployment, monitoring, and testing. It explains what was configured, why that configuration was needed, and how the selected AWS services worked together to make the application function correctly. It also explains the NLP concepts used in the Lambda backend, the main challenges faced during implementation, and how those issues were resolved during the project.

The project was completed in the **AWS Academy Lab** environment, in the lab titled **AWS Academy Lab Project - Cloud Web Application Builder**. The lab was accessed through AWS Canvas, clicked start lab and waited for the lab status indicator turned green, which showed that the environment was active and ready to use.

After opening the AWS Management Console from the lab, the selected region was set to **us-east-1 (N. Virginia)**. Then the implementation was started in this Lab.

Concepts used in the project:

Security:

The main security concepts used in this project were SSE-S3 encryption, HTTPS delivery through CloudFront, CORS configuration for frontend backend communication, and role based access through the Lambda execution role. These were important because the application was not just a static page sitting alone. It involved a frontend hosted in S3, a backend running in Lambda, JSON result storage in S3, and communication between the browser and the backend through the Lambda Function URL. Because of that, security had to be handled both at the storage level and at the communication level.

SSE-S3:

One important security feature used was Server-Side Encryption with Amazon S3 managed keys (SSE-S3). This was enabled in the Properties tab under Default encryption for the S3 bucket. The purpose of this configuration was to make sure that the objects stored inside the bucket were encrypted at rest. This means that when files are stored in S3, AWS automatically encrypts them before saving them physically in storage and decrypts them when they are accessed by an authorized request. This was important as S3 bucket was not only used for static website hosting, but was also used to store the JSON analysis output generated by the Lambda backend. Even though the frontend files had to remain publicly readable for website hosting, enabling SSE-S3 still added a basic storage-level security measure for the bucket.

CloudFront / HTTPS:

Another important security feature in the project was using Amazon CloudFront to deliver the frontend over HTTPS. Instead of relying only on the S3 website endpoint, CloudFront was placed in front of the S3-hosted frontend so the website could be accessed through an HTTPS URL. HTTPS encrypts the traffic between the user's browser and the website, which helps protect data in transit. It makes the website connection more secure by using TLS encryption and prevents the frontend from being delivered over less secure HTTP in the final deployment step.

CORS:

Another important security concept that used during implementation was CORS, or Cross-Origin Resource Sharing. Since the frontend website and the backend Lambda Function URL were being accessed from different origins, the browser had to be given permission to allow that communication. This was configured through the Lambda Function URL CORS settings so the frontend JavaScript could send requests to the backend. CORS was not just a technical detail, but one of the main security controls for browser-based backend access in this project.

Authorization roles / permissions:

Another security concept used in the project was role-based access through the Lambda execution role. The Lambda function was created using the existing LabRole, which allowed the backend to run inside the AWS Academy environment and interact with AWS services such as S3 and CloudWatch without manually hardcoding broader permissions inside the application code. As service access was being controlled through AWS role-based permissions rather than being handled in an unsafe or overly open way. It also kept the backend aligned with the controlled AWS lab environment used for the project.

Natural Language Processing:

Natural Language Processing, or NLP, is the part of AI computing that focuses on how text and language can be read, processed, and understood by a program. It allows a system to take normal text written by a user and turn it into something meaningful, such as identifying the tone of the text, summarizing the text, finding important words or names, or checking how easy the text is to read. In this project, the NLP part was used to analyze the text entered on the website and return structured results instead of just treating the input as plain text.

NLP part is contained in Appendix B, which is the provided Python Lambda function code. The Lambda function is built around four main tasks: sentiment analysis, entity extraction, key phrase extraction, and text statistics/readability analysis. Instead of using a large external NLP library, the code handles these tasks using its own Python logic with predefined word lists, regex patterns, and text-processing rules.

The sentiment analysis works by taking the input text, converting it into lowercase words, and comparing those words against two predefined sets: positive words and negative words. Words such as good, great, excellent, and love are counted as positive, while words such as bad, terrible, awful, and hate are counted as negative. The function then calculates positive, negative, and neutral scores based on how many of those words appear in the text. After that, it decides whether the overall sentiment is POSITIVE, NEGATIVE, MIXED, or NEUTRAL. It also returns the matched positive and negative words along with percentage-style scores and a confidence value.

The entity extraction uses predefined regex patterns to search the text for things such as names, emails, URLs, phone numbers, dates, numbers, and organization related words. Then it checks the text against a built in list of known locations, such as cities, states, and countries. When a match is found, the function stores the entity text, its type, its position in the input, and a simple confidence value.

The key phrase extraction works by splitting the text into sentences and then building short multi-word phrases from those sentences. It removes punctuation, ignores very common filler words, and tries to keep phrases that are more meaningful. The phrases are then scored based on their length and returned as the top key phrases with confidence values.

The text statistics and readability part calculates values such as word count, sentence count, average words per sentence, and character count. It also estimates syllables and uses that information to calculate a Flesch Reading Ease style score. Based on that score, the text is labeled with a readability level such as Very Easy, Standard, Difficult, or Very Difficult. The code also finds the most frequent non-common words in the text and returns them as top words.

Appendix A, Frontend Files:

Before beginning of phase 2, the following frontend files had to be created:

index.html

The index.html file was created as the main frontend structure of the application. It defines the layout of the webpage, including the title area, the text input box, the Analyze button, the status message section, and the result sections for sentiment, entities, key phrases, and readability. This file acts as the base of the website and gives the frontend a place to display both user input and backend results.

style.css

The style.css file was created to control the visual design of the frontend. It was used to style the page layout, fonts, colors, spacing, input box, button, and results section so the website looked clean, organized, and readable. This file made the frontend easier to use and helped present the returned analysis results in a clear way.

script.js

The script.js file handled the main frontend logic and took more work because it was responsible for connecting the website to the backend. This file stored the Lambda Function URL, collected the text entered by the user, sent it to the backend using a POST request, and then handled the JSON response returned by Lambda. It also controlled how the results were shown on the page by updating the sentiment, entities, key phrases, and text statistics sections dynamically. It also managed feedback such as showing status messages, hiding the results section before analysis, and displaying it only after a successful response. It made the frontend interactive and allowed the webpage to actually communicate with the backend instead of only acting as a static page.

Appendix B, Lambda function:

lambda_function.py:

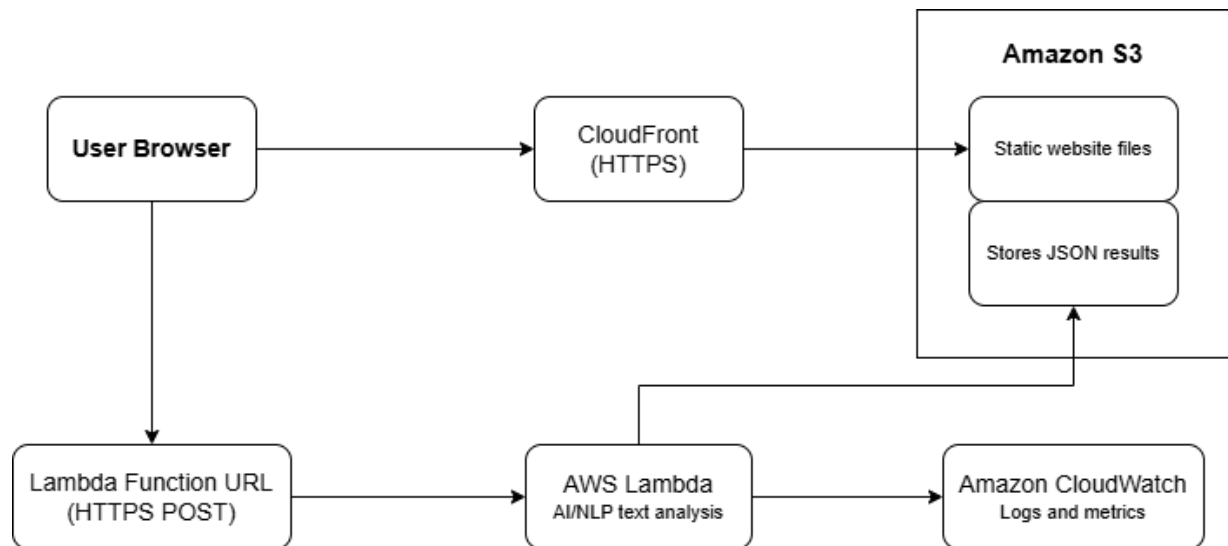
The Lambda function code was the backend code that was already provided for this project. This contained the NLP part of the project. This Python code receives the text sent from the frontend, processes it, and returns the analysis results back to the website. Inside the function, separate logic is used for sentiment analysis, entity extraction, key phrase extraction, and readability scoring. After the text is processed, the function creates a structured JSON response containing all of those results. The Lambda code also stores that output in Amazon S3 as a JSON file, which allows each request result to be saved in the results/ folder. In addition to processing requests, the function also sends execution logs and metrics to CloudWatch, which made it possible to monitor backend activity during testing and deployment.

Implementation:

Phase 1- Architecture Diagram & Cost Estimate:

Architecture and planning:

The architecture planning was done before the AWS implementation so the full flow of the application could be understood. The architecture diagram was created in draw.io based on the requirements. Architecture diagram shows how the main AWS services connect to each other. The services for the project include the User Browser, AWS CloudFront, Amazon S3, AWS Lambda and Amazon CloudWatch.



The flow of this project starts when the user opens the website in a browser. The frontend of the website is stored in Amazon S3 and delivered through CloudFront using HTTPS. On the webpage, the user enters some text and clicks the Analyze button. That text is then sent from the frontend to the backend using the Lambda Function URL through an HTTPS POST request. AWS Lambda receives the request and runs the text analysis logic. The Lambda function checks the text and generates results such as sentiment, entities, key phrases, and readability statistics. After the analysis is complete, the results are sent back to the frontend so they can be shown on the website for the user. At the same time, the Lambda function also stores the output in Amazon S3 as a JSON file, so the analysis results are saved for later use. While all of this is happening, Amazon CloudWatch collects logs and metrics from the Lambda function, which helps monitor performance, execution activity, and any errors. In this way, it connects the browser, frontend hosting, backend processing, storage, and monitoring into one complete serverless workflow.

This architecture is simple, secure, and cost effective. It uses managed AWS services like S3, CloudFront, Lambda, and CloudWatch, so there is no need to manage servers manually. It combines static website hosting, serverless processing, storage, and monitoring in one clean design.

Cost Estimate / Analysis:

The cost estimate for this project was prepared in the AWS Pricing Calculator during the planning stage by adding the main services used in the application, which were **Amazon S3**, **AWS Lambda**, **Amazon CloudFront**, and **Amazon CloudWatch**. Minimal usage values were entered for each service because the project was small and only needed a basic level of traffic, storage, and monitoring. The purpose of this step was to understand the expected cost of the deployment before building the application and to make sure the design stayed within a very low cost range. Based on the calculator result, the total estimated cost came out to **\$0.00** under the minimal usage and free-tier assumptions used for the project. This is because the application only used lightweight static hosting, serverless processing, simple logging, and limited storage of JSON output files.

Phase 2 - Create S3 Bucket & Host Static Website:

In this step, an **Amazon S3 bucket** was created to host the frontend of the application as a static website. A new bucket named **cs524-haseeb-website** was created in the **US East (N. Virginia)** region. This bucket shall later store the frontend files of the application ie. `index.html`, `style.css`, and `script.js`.

During bucket creation, “**Block all public access**” was **unchecked** and the acknowledgment box completed. This makes frontend reachable publicly from a browser, and if public access stayed blocked, the static website would not load correctly.

After the bucket was created, the **Properties** tab was opened and **Static website hosting** was enabled. In that section, the **Index document** was set to `index.html`, and the **Error document** was also set to `index.html`. This is done as S3 needed to know which file should open first when the website URL is visited, and which file should be returned if a page path does not directly match another file. Once this was enabled, the **bucket website endpoint URL** was generated. That endpoint later became the direct link used to test the website before CloudFront was added.

The next step was done in the **Permissions** tab, where **the given bucket policy was added under the Bucket policy** section. The policy allowed public `s3:GetObject` access to the files stored inside the bucket. This step was needed because enabling static website hosting was not enough. Even with website hosting turned on, the browser still would not be able to read and load the frontend files unless the objects inside the bucket had permission to be accessed publicly. The bucket policy is allowing the website files to be read from the browser.

After that, the **Properties** tab was opened again and **Default encryption** was checked. In that section, **Server-side encryption with Amazon S3 managed keys (SSE-S3)** was already enabled. This so the bucket would still follow a basic AWS security practice by encrypting data stored inside S3 at rest. Even though the frontend files had to be publicly readable for hosting purposes, enabling encryption added a layer of protection for the stored objects inside the bucket.

The frontend files **`index.html`**, **`style.css`**, and **`script.js`** were prepared locally but were not uploaded yet. This was done because the backend **Lambda Function URL** had not been created at that point, and `script.js` still needed that URL before the frontend could communicate with the backend correctly.

This phase successfully prepared the frontend hosting side of the project. The S3 bucket was created, public access was configured correctly, static website hosting was enabled from the **Properties** tab, the bucket policy was added from the **Permissions** tab, the website endpoint was generated, and SSE-S3 encryption was confirmed.

S3 Endpoint URL: <http://cs524-haseeb-website.s3-website-us-east-1.amazonaws.com>

Phase 3 - Create Lambda Function (AI Backend):

In this step, the backend of the application was created using **AWS Lambda** so the text analysis could run in a serverless way without setting up a separate server. A new Lambda function named **cs524-ai-analyzer** was created in the Lambda console using the **Python 3.12** runtime and the existing **LabRole** execution role. This setup was needed because the backend logic for the project was written in Python, and the function also needed permission to interact with AWS services.

After the function was created, the **Configuration** section was opened and the **General configuration** settings were edited. In that area, the timeout was changed from **3 seconds** to **30 seconds**. This change was done because the function was not only receiving text and returning a response, but also running multiple NLP operations such as sentiment analysis, entity extraction, key phrase extraction, and readability scoring before storing the output in S3. Keeping the timeout at 3 seconds could have caused the function to stop too early if processing or storage took longer than expected. Increasing the timeout gave the function enough time to complete the full backend workflow properly.

The provided Python backend code was then pasted into the the Lambda code editor. In that code, the `BUCKET_NAME` variable was updated to **cs524-haseeb-website** so it matched the actual S3 bucket created earlier. The code handled the full text analysis flow by receiving input text, checking the sentiment, extracting entities, identifying key phrases, calculating readability statistics, and packaging all of that into a structured response. The code was **deployed**.

After the code was deployed, the **Test** section in the Lambda console was used to create a test event. A given sample JSON input containing text and the HTTP method was added in the event body. This test step was important because it allowed the backend to be checked directly inside AWS before connecting it to the website. Instead of debugging both the frontend and backend at the same time, this made it possible to confirm first that the Lambda function itself was working correctly on its own. When the test was run, the function returned a successful **statusCode: 200**, which showed that the input was being received properly, the text was being analyzed correctly, and the function was returning a valid output response.

Once the backend test was successful, a **Lambda Function URL** was created for the function. This step was needed because the frontend website had to communicate with the backend through a reachable HTTPS endpoint. Without the Function URL, the website would have no direct way to send the text entered by the user to the Lambda function. During this setup, the **Auth type** was set to **NONE** and **CORS** was enabled so browser requests from the frontend could be accepted. This was important because the frontend website and the backend endpoint were being accessed through different URLs, so browser-side access settings had to allow that communication. The Function URL later became the value that was inserted into `script.js` in the next phase so the frontend could call the backend correctly. In the CORS configuration, **Allow origin** was set to `*`, in **Allow methods** all methods were added, and **Allow headers** included **content-type**.

Once this configuration was completed, the Function URL was saved and copied for further use.

Lambda function URL: <https://wqjynsr74tock6q4pd33kg3meu0mzkde.lambda-url.us-east-1.on.aws/>

Phase 4 - Upload Website & Connect to Lambda:

In this step, the frontend of the application was connected to the backend so the website could send user input for analysis and display the returned results. The first part of this phase was updating the **script.js** file by replacing the placeholder value with the **Lambda Function URL** that had been created in the previous phase.

After the backend URL was added into script.js, the three frontend files **index.html**, **style.css**, and **script.js** were uploaded into the S3 bucket that had already been prepared for static website hosting. These files together formed the full frontend of the application. The HTML file provided the structure of the page, the CSS file controlled the visual design, and the JavaScript file handled the actual communication with the backend.

The next step was opening the website through the **S3 website endpoint** that had been generated earlier in Phase 2. This was done to test the static website directly before placing CloudFront in front of it. Once the page loaded, sample text was entered into the text box and the **Analyze** button was clicked.

There was an error at this point, the browser showed a **“Failed to fetch” error** even though the Lambda Function URL itself was reachable. The issue turned out to be caused by **duplicate CORS headers**. CORS had already been enabled in the Lambda Function URL settings, but the Lambda Python code was also manually returning Access-Control-Allow-Origin, Access-Control-Allow-Headers, and Access-Control-Allow-Methods. Because of that, the response contained duplicate values such as *, *, and the browser rejected it. More detailed Explanation on the debugging shall be provided in the **Challenges faced and Fixes** section of the report. The fix was to edit one line in the Lambda code ie. remove the CORS headers from the Lambda code and let the Function URL CORS settings handle browser access. After redeploying the Lambda function and refreshing the S3-hosted website, the Analyze feature **started working correctly**.

At that point, the frontend JavaScript sent the text to the Lambda Function URL, and the backend processed the request. The **returned output included sentiment, entities, key phrases and text statistics/readability** which were then displayed on the webpage. This step showed that the frontend and backend were now working together correctly through the browser.

After the website was able to analyze text successfully, the S3 bucket was checked again to verify that the backend was also storing the results properly. Inside the **results/** folder of the bucket, JSON output files were created for each analysis request. This showed that the Lambda function was not only returning the results back to the browser, but was also saving them in S3 exactly as intended. This storage step was important because it gave a persistent record of the backend output and confirmed that the integration between the frontend, Lambda, and S3 was working end to end.

The phase successfully completed.

Phase 5 - CloudFront HTTPS Distribution:

In this step, **Amazon CloudFront** was set up so the frontend website could be delivered securely over **HTTPS** instead of only being accessed through the direct S3 website endpoint. The **CloudFront** service was opened in the AWS console, the free tier option was not available to choose so **the pay as you go option was selected** and a new distribution was created.

While creating the distribution, the **S3 bucket** that was already hosting the frontend files was selected as the **origin**. Since static website hosting had already been enabled on the bucket, the **website endpoint** was used for the **origin configuration**. This step was needed because the frontend files were already being served correctly from the S3 website endpoint, so CloudFront had to point to that same working source. **The origin path was left blank**, and the **recommended settings were used for origin and cache configuration**.

During the CloudFront distribution setup, a **security protections** screen appeared. That option was left **disabled** as extra protection services were not needed for this project.

After the distribution was created, time was given for CloudFront to finish deploying until the status changed to **Enabled**. CloudFront URL would not work properly until the deployment process was complete across the CloudFront network. Once the distribution became active, the **Default root object** setting was updated to **index.html**. This was needed so that when the CloudFront URL was opened, the application would automatically load the main frontend page instead of showing an error or directory-style response. Without setting the default root object, the website might not have opened correctly from the root of the CloudFront domain.

The next step was testing the application through the new **CloudFront HTTPS URL**. The CloudFront domain name was copied, opened in the browser, and used to load the website. After the site opened successfully, sample text was entered into the input box and the **Analyze** button was clicked again to verify that the application still worked correctly even after CloudFront had been added in front of the S3 frontend. This step confirmed that secure delivery through CloudFront did not break the frontend-backend communication and that the application could still send text to the Lambda backend and display the returned results properly.

If an older cached version of the frontend had appeared, a CloudFront invalidation using **/*** would have been the correct fix.

This phase was successful and added a secure delivery layer to the frontend side of the project.

CloudFront URL: diq0ysy4we8cm.cloudfront.net

Phase 6 - CloudWatch Monitoring

In this step, **Amazon CloudWatch** was used to monitor the application after the website and Lambda integration were already working.

The first part of this phase focused on the **log groups** section in CloudWatch, where the log group for the Lambda function `/aws/lambda/cs524-ai-analyzer` was opened. **Inside that log group, log streams were checked to confirm that Lambda executions were being recorded properly.** This showed that the backend requests were actually reaching the Lambda function and being processed. It also made it possible to inspect execution details, troubleshoot issues, and verify that the backend was running when text was submitted from the website.

Next, a **CloudWatch dashboard** was created so the Lambda function could be monitored visually instead of only through raw log entries. A dashboard named **CS524-AI-Dashboard** was created, and widgets were added for **Invocations, Errors, and Duration**. These metrics were chosen because they gave a clear picture of how often the backend was being used, whether any failures were happening, and how long the function was taking to complete. Logs are useful for detailed troubleshooting, but a dashboard makes it much easier to monitor the health and activity of the application in a simple visual format.

The next part of this phase focused on creating a **CloudWatch alarm** based on the **Lambda Errors** metric. In the alarm setup, the **Sum** statistic was used, the **period** was set to **5 minutes**, and the **threshold was configured** so the alarm would trigger if the number of errors went above **5**. This was done so the system could alert when the backend started failing repeatedly instead of requiring manual checking all the time. This step was useful because it showed how backend failures could be monitored automatically rather than only being discovered after manually reviewing logs or dashboards.

After that, an **SNS topic** was created and connected to the alarm so email notifications could be sent when the alarm condition was triggered. **My email address was entered** during the SNS setup, and **the subscription was confirmed through the confirmation email**. The alarm by itself would only exist inside AWS, while SNS made it possible to send a direct notification outside the console. That made the monitoring setup more realistic, since alerts could now reach the user without having to constantly stay inside the AWS dashboard and watch for changes manually.

The final part of this phase involved **generating more real activity by using the website several times** and then returning to the **CloudWatch dashboard to verify that the widgets displayed live data**. Multiple analysis requests were submitted from the website so the Lambda function would generate new logs, increase invocation counts, and update timing information. This confirmed that the monitoring setup was connected properly to the working application.

This phase successfully completed the monitoring side of the project.

Deleting Everything:

After the project is graded, all AWS resources created during the implementation will be deleted as part of the cleanup process. This includes disabling and deleting the **CloudFront distribution**, emptying and deleting the **S3 bucket** used for static website hosting and JSON result storage, deleting the **Lambda function** used for backend text analysis, and removing related **CloudWatch resources** such as dashboards, alarms, and log groups.

Challenges faced and how they were resolved:

Hard time making the frontend files especially in JS:

The frontend side of the project took more time than expected. The HTML had to be structured in a way that could take user input clearly and also provide sections for sentiment, entities, key phrases, and readability results. The CSS wasn't as bad as I thought and didn't take much time.

Most of the time was actually spent on the **JavaScript** side. That part had to handle the full request and response flow between the browser and the Lambda backend. The entered text had to be collected, sent through a POST request to the Lambda Function URL, and then the returned JSON response had to be parsed and displayed in the correct sections of the page.

A lot of checking was needed in JavaScript because the field names returned by Lambda, such as sentiment, entities, key phrases, and text statistics, had to line up with the correct result containers on the page. Logic also had to be added to hide the results section before analysis, show loading or status messages while processing, and then display the results only after a successful response came back. If any element ID in the HTML did not match the JavaScript, or if the expected JSON structure did not line up properly, the page would fail even if the backend itself was running correctly. This part took repeated testing and refinement before the full website started behaving the way it was supposed to.

Phase 4 - "Failed to fetch" error due to Lambda function:

During Phase 4, one major issue came up when clicking Analyze from the hosted website but the browser showed a "Failed to fetch" At first, the issue looked like it might be coming from the frontend JavaScript, because the website was loading but clicking Analyze kept showing a "Failed to fetch" error.

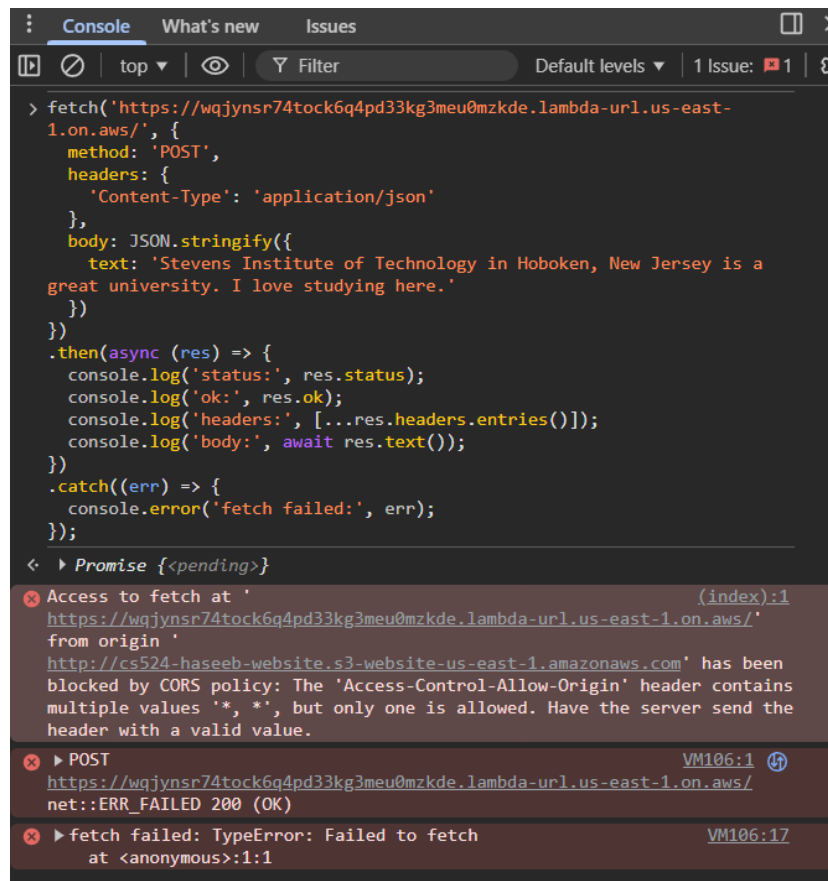
The JavaScript was checked first to make sure the request was being sent to the correct backend endpoint, and the Lambda Function URL inside script.js was verified more than once. After that, the Lambda side was checked as well.

The Function URL settings were reviewed, the backend endpoint opened correctly in the browser, and the Lambda test inside AWS was still returning a successful statusCode: 200, which showed that the function itself was running and the backend logic was working. Since the URL, the Lambda configuration, and the Lambda test all looked correct, I opened the browser console to get a more detailed error message from the hosted website. That console output showed that the request was actually reaching the backend, but the browser was rejecting the response because there was a problem with the response headers.

After checking the details more closely, the actual problem was traced to the CORS headers inside the Lambda function. The original Python code had a headers dictionary that manually returned Access-Control-Allow-Origin, Access-Control-Allow-Headers, and Access-Control-Allow-Methods in the Lambda response. At the same time, CORS had already been enabled in the Lambda Function URL settings, which meant AWS was also adding those headers automatically. Because of that, the final response contained duplicate header values such as *, *, and the browser blocked the response even though the backend had executed successfully.

The change made in the Lambda function was to remove those manually returned CORS headers and replace the original header dictionary with an empty one, so AWS Function URL CORS could handle that part by itself. After redeploying the Lambda function and refreshing the hosted website, the Analyze feature started working correctly. This issue showed that the backend can appear fully correct when tested by itself, but browser integration can still fail.

Debugging in console:



```
> fetch('https://wqjynsr74tock6q4pd33kg3meu0mzkde.lambda-url.us-east-1.on.aws/', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    text: 'Stevens Institute of Technology in Hoboken, New Jersey is a great university. I love studying here.'
  })
})
.then(async (res) => {
  console.log('status:', res.status);
  console.log('ok:', res.ok);
  console.log('headers:', [...res.headers.entries()]);
  console.log('body:', await res.text());
})
.catch((err) => {
  console.error('fetch failed:', err);
});
```

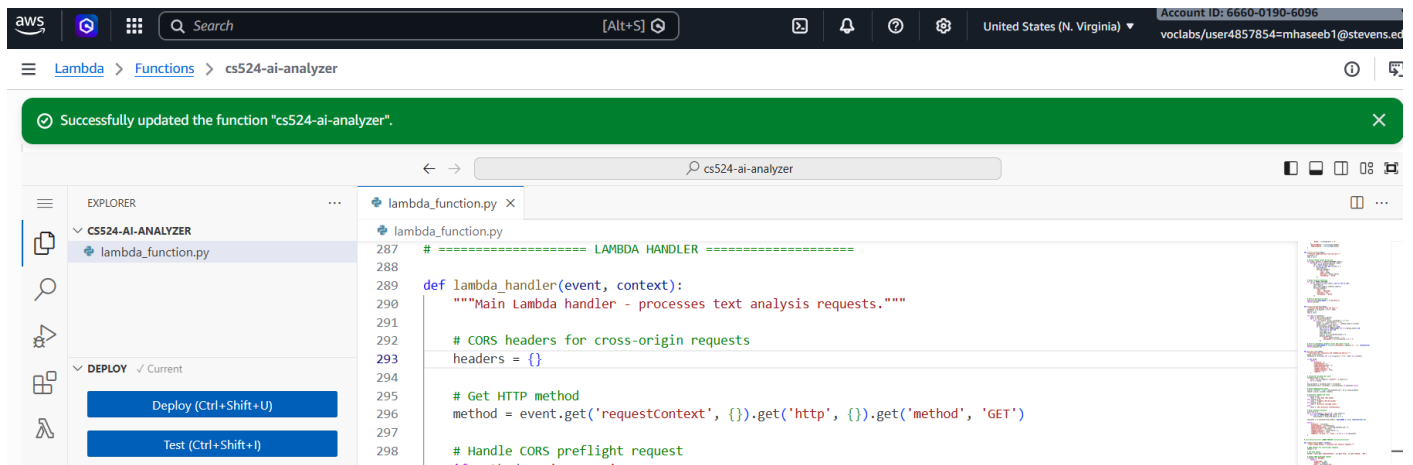
< ▶ Promise {<pending>}

✖ Access to fetch at 'https://wqjynsr74tock6q4pd33kg3meu0mzkde.lambda-url.us-east-1.on.aws/' from origin 'http://cs524-haseeb-website.s3-website-us-east-1.amazonaws.com' has been blocked by CORS policy: The 'Access-Control-Allow-Origin' header contains multiple values '*', '*', but only one is allowed. Have the server send the header with a valid value.

✖ ▶ POST https://wqjynsr74tock6q4pd33kg3meu0mzkde.lambda-url.us-east-1.on.aws/ net::ERR_FAILED 200 (OK) VM106:1

✖ ▶ fetch failed: TypeError: Failed to fetch at <anonymous>:1:1 VM106:17

CORS Header changed and deployed:



Phase 6 - CloudWatch widget not Updating:

In phase 6, A minor problem came up in CloudWatch when the Invocations metric on the dashboard did not seem to update at first after sending multiple analysis requests. The issue was not with Lambda execution itself, but with the way the dashboard widget had been configured. The correct Lambda function, region, time range, and metric view had to be checked and adjusted so the widget would display the real invocation data. After correcting the dashboard settings and generating more website requests, the dashboard started showing the invocation data properly. This helped confirm that the monitoring setup was working as expected.

Lessons Learned:

This project helped show how different AWS services connect together to build one complete cloud application. An understanding was developed around how **Amazon S3**, **CloudFront**, **AWS Lambda**, and **CloudWatch** depend on each other in the full workflow. Static website hosting in S3 handled the frontend files, CloudFront added secure HTTPS delivery, Lambda handled the backend processing, and CloudWatch made it possible to monitor logs, invocations, errors, and execution behavior.

Another important lesson came from **developing and deploying a static website** in the cloud. Hosting the frontend in S3 looked simple at first, but it required several steps to work properly, such as public access configuration, bucket policy, static website hosting, and frontend file upload. The website also had to be connected carefully with the Lambda backend through JavaScript so that the text entered by the user could be sent for analysis and the returned results could be displayed correctly.

The project also helped strengthen understanding of **security**, there was a clearer understanding of encryption in S3, HTTPS delivery through CloudFront, and correct CORS handling between the frontend and backend.

The project also helped strengthen understanding of **NLP**, showed how text can be analyzed for sentiment, entities, key phrases, and readability.

The troubleshooting part was also a major lesson, especially when the CORS issue caused the website to fail even though the backend itself was working.

Conclusion:

This project successfully combined the main AWS components needed to build a complete cloud-based web application. **Amazon S3** was used to host the frontend files as a static website, **Amazon CloudFront** was used to deliver the site securely over HTTPS, **AWS Lambda** was used as the backend service to process the user input, and **Amazon CloudWatch** was used for logging, monitoring, and alarms.

The frontend accepted text from the user, sent that text to the Lambda backend through the **Lambda Function URL**, displayed the returned analysis results on the webpage, and also allowed the backend to store those results in **Amazon S3** as JSON files.

All of these components were connected properly, and the full application worked successfully from browser input to backend response and storage output.

The project also helped strengthen practical understanding of how cloud services work together in a real deployment. Static website hosting, secure delivery, serverless processing, monitoring, and storage were not just studied separately, but were implemented as part of one system. Security concepts such as **SSE-S3 encryption**, **HTTPS delivery**, and **CORS handling** became much clearer through implementation. The text analysis logic in Lambda also gave practical exposure to NLP-related processing such as sentiment, entities, key phrases, and readability.

Links:

CloudFront URL: diq0ysy4we8cm.cloudfront.net

S3 endpoint URL: <http://cs524-haseeb-website.s3-website-us-east-1.amazonaws.com>